DEEP LEARNING ARCHITECTURE

Understanding Transformers

A Beginner's Guide

> From Sequence Processing to Attention Mechanisms:
> A Comprehensive Journey Through Modern NLP

Deep Neural Networks | Natural Language Processing

What are Sequence-to-Sequence Networks?

The foundation of modern language models

Core Concept

A seq2seq network takes a series of inputs and generates a series-based output. It consists of two main components:

Encoder
Maps input into an encoding scheme

Decoder
Decodes to determine desired output

Why It Matters

Seq2seq networks revolutionized how machines process sequential data, enabling applications from translation to speech recognition.

Three Main Types

One-to-Many

Fixed → Variable

Example: Image Captioning

Takes a fixed-size vector (image) and generates a sequence of any length (description)

Many-to-One

Variable → Fixed

Example: Sentiment Analysis

Takes variable input (sentence) and generates fixed-size output (positive/negative)

Many-to-Many

Variable → Variable

Example: Machine Translation

Takes variable input (English sentence) and generates variable output (Urdu sentence)

The Problem with RNNs and LSTMs

Why we needed a new architecture

Problem 1

Slow Training

RNNs and LSTMs process data **sequentially** - one token at a time. This sequential dependency means they cannot parallelize operations.

Impact: Training takes much longer, especially with large datasets

Problem 2

Vanishing/Exploding Gradients

With **long sequences**, gradients become extremely small (vanish) or extremely large (explode) during backpropagation.

Impact: Model fails to learn long-range dependencies

LSTM's Solution

LSTMs solve the gradient problem with gating mechanisms

But... slow training persists!

The Need for Transformers

We needed an architecture that could **process all tokens in parallel** while capturing **long-range dependencies** effectively.

Introducing Transformers: A New Approach

The breakthrough that changed everything

Key Innovation

Transformers are non-recurrent models. Unlike RNNs, they don't rely on sequential processing.

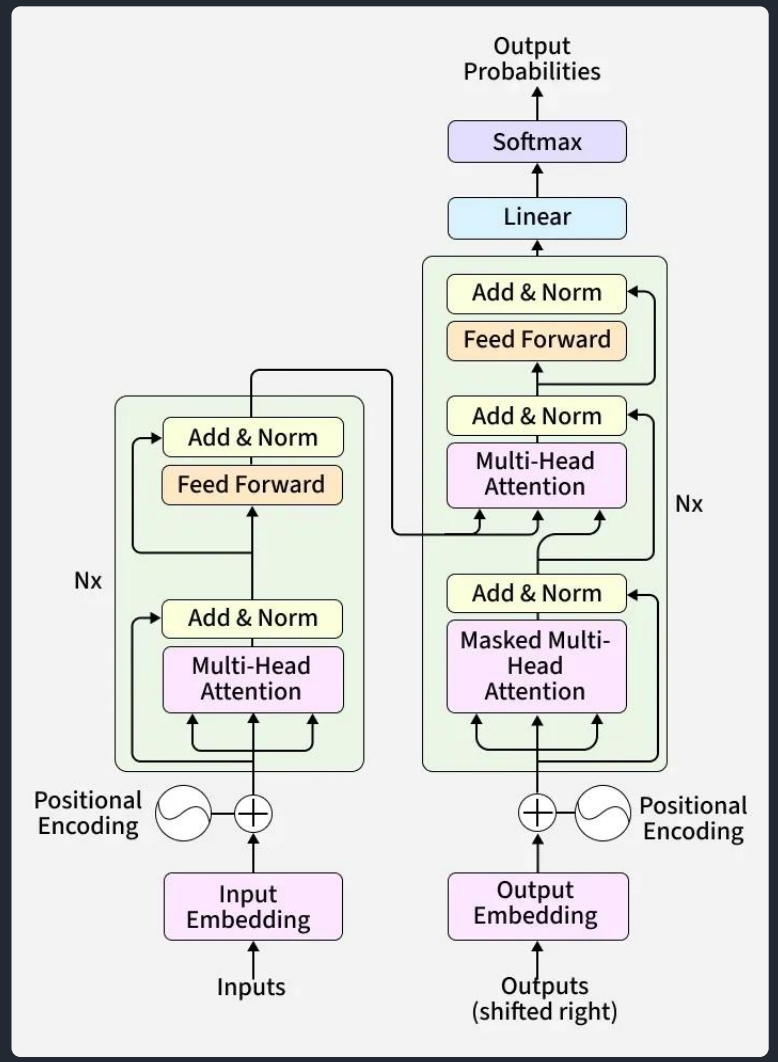
Revolutionary: All tokens can be processed in parallel, dramatically improving training efficiency!

Architecture Overview

6× Encoder Stack
Six identical encoder units stacked together

6× Decoder Stack
Six identical decoder units stacked together

Transformer Architecture



The original Transformer architecture from "Attention Is All You Need"

The Encoder: Breaking It Down

Understanding the input processing pipeline

1 Positional Encoding

Since Transformers process tokens in parallel, they need to know **where each word is** in the sentence.

Input: Word embeddings → **Output:** Positional embeddings

2 Multi-Head Attention

Generates **attention vectors** (weights) that show how much each word relates to every other word.

Based on **scaled dot-product attention**

3 Feed Forward

Transforms attention vectors into a format acceptable for the next encoder block or decoder.

Encoder Flow: English → Urdu

Input

"I love learning" (English sentence)

↓

Embeddings + Positional Encoding

Each word becomes a vector with position info

↓

6 Encoder Units

Each applies multi-head attention + feed forward

↓

Output

Contextualized representations for decoder

The Decoder: How It Works

Generating output one word at a time

1 Masked Multi-Head Attention

Generates attention vectors for output words, but with **masking** to hide future words.

Why mask? Prevents decoder from "cheating" by seeing future words

2 Encoder-Decoder Attention

Connects encoder and decoder by computing attention between **input (English)** and **output (Urdu)** sequences.

Enables decoder to focus on relevant input words

3 Feed Forward Block

Transforms attention vectors into format suitable for next decoder block or output layer.

4 Linear Layer

Expands dimensions to match the size of the output vocabulary (number of unique words in target language).

5 Softmax Layer

Converts outputs into **probability distributions** across all possible words.

The word with **highest probability** is selected

Decoder's Goal

The decoder runs iteratively, generating one word at a time, until it produces the end-of-sentence token.

Key Insight: During training, the decoder receives the correct previous words (teacher forcing). During inference, it uses its own predictions.

Positional Encoding: Giving Words Their Place

Why order matters in parallel processing

Why Do We Need It?

Transformers process all tokens **in parallel**, so they don't inherently know word order like RNNs do.

Example:

"AJ's **dog** is a cutie" [position 2]

"AJ looks like a **dog**" [position 5]

Same word, different meanings based on position!

The Mathematics

Uses **sine and cosine** functions at different frequencies:

$$PE(pos,2i) = \sin(pos / 10000^{(2i/d)})$$

$$PE(pos,2i+1) = \cos(pos / 10000^{(2i/d)})$$

- **pos:** position in sequence
- **i:** dimension index
- **d:** embedding dimension
- **10000:** scaling factor

How It Works

We add a **positional encoding vector** to each word embedding, giving the model position information.

Final Input = Embedding + Positional Encoding

Key Benefits

- **Unique encoding:** Each position gets a distinct vector
- **Relative positions:** Model learns distance between words
- **Generalization:** Works for sequences longer than training

Intuition: Think of it like giving each word a "coordinate" in the sentence, so the model knows both **what** the word is and **where** it is.

Understanding Attention: The Core Mechanism

How Transformers know what to focus on

The Web Search Analogy

Attention works like a **web search**:

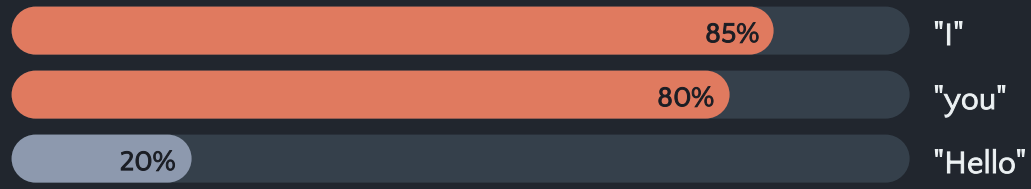
Q **Query (Q)**
What you're looking for (your search query)

K **Key (K)**
Index terms, titles, descriptions (what's available)

V **Value (V)**
The actual content you want (search results)

Example: "Hello I like you"

A trained self-attention layer associates the word **"like"** with:



Subject-verb-object relationship captured!

Self-Attention

A mechanism where a sequence **attends to itself**. Each word learns attention weights for all other words in the same sequence.

Why It's Powerful

- ✓ Captures **contextual relationships**
- ✓ Handles **long-range dependencies**
- ✓ Works in **parallel**

Attention Definition: A mapping from a **query** and a set of **key-value** pairs to an output, where query, keys, values, and output are all vectors.

Scaled Dot-Product Attention

The mathematical heart of Transformers

The Formula

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

Why Scale by $\sqrt{d_k}$?

When Q and K have dimension d_k, their dot product has **variance = dk**.

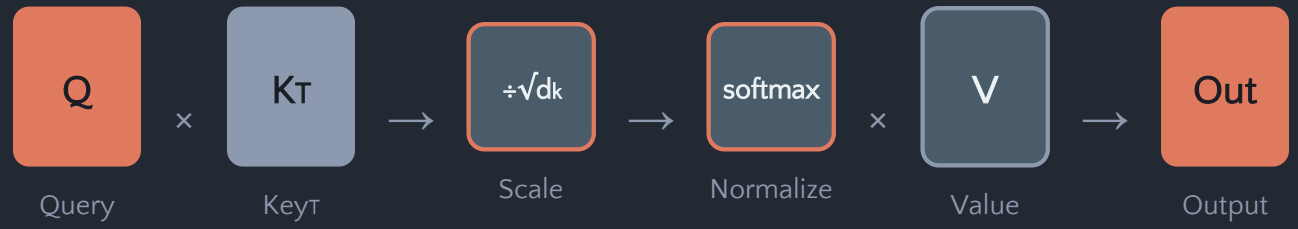
The Problem: Large values push softmax into regions with very small gradients.

The Solution: Scaling by $\sqrt{d_k}$ keeps variance ≈ 1, ensuring stable gradients.

Step-by-Step Process

1. **Compute:** Q × K^T
2. **Scale:** Divide by $\sqrt{d_k}$
3. **Normalize:** Apply softmax
4. **Weight:** Multiply by V

Visual Representation



Key Insight: Each token produces its own Q, K, V vectors, allowing the model to learn different representations for different purposes.

Multi-Head Attention: Multiple Perspectives

Why one attention head isn't enough

Limitation of Single-Head

Single-head attention learns only **one type of relationship** at a time.

But language has multiple relationships:

- **Syntactic:** subject-verb agreement
- **Semantic:** meaning and context
- **Positional:** nearby vs distant words

The Solution

Multi-head attention runs multiple attention mechanisms in parallel, each focusing on different aspects.

How It Works

1. **Project Q, K, V multiple times** using different learned linear projections

$\text{head}_i = \text{Attention}(QW_{Qi}, KW_{Ki}, VW_{Vi})$

2. **Each head operates independently**, capturing different relationships

Head 1	Head 2	Head 3
Grammar	Long-range	Local context

3. **Concatenate all heads** and apply final linear transformation

Benefits

- **Richer representations:** Multiple perspectives combined
- **Parallel processing:** All heads computed simultaneously
- **Better positional info:** Different heads attend to different segments

Original Paper: 8 attention heads, each with $d_k = d_v = 64$, giving total dimension of 512

Training Transformers

How the model learns to translate

1 Input Preparation

Convert words to embeddings (e.g., word2vec), add positional encoding, feed to encoder.

2 Encoder Processing

Stack of 6 encoders processes input, producing contextualized representations.

3 Decoder Input

Feed target sentence (e.g., Urdu) shifted right by one position with start token.

4 Decoder Processing

Stack of 6 decoders uses encoder output and decoder input to generate predictions.

5 Output Generation

Linear layer + softmax produce probability distribution over vocabulary.

6 Loss Calculation

Compare predictions with actual target words using cross-entropy loss.

Teacher Forcing

During training, decoder receives **correct** previous words, not its own predictions.

Why? Helps model learn faster by preventing error accumulation

→ Why Shift Right?

Prevents model from learning to **copy** the decoder input. Forces it to **predict** the next word.

Training Data: Pairs of sentences in both languages (e.g., English-Urdu translations)

Inference: Making Predictions

How the model translates new sentences

How It Differs from Training

During inference, the model **doesn't know** the correct output. It must generate words one at a time.

No teacher forcing! Decoder uses its own predictions as input for next step.

The Iterative Process

1. Input English sentence + token
2. Generate first word (highest probability)
3. Append first word to decoder input
4. Generate second word
5. Repeat until token

Example: English → Urdu

Input: "I love learning"

Step 1: → "میں"

Step 2: میں → "محبت"

Step 3: ... میں محبت → "کرتا"

Step 4: ... محبت کرتا → "ہوں"

Step 5: ... کرتا ہوں →

Error Accumulation

If the model makes a mistake early, that error can propagate to subsequent predictions. This is why training with teacher forcing is crucial!

Key Difference: Training uses ground truth; Inference uses model predictions

Residual Connections & Layer Normalization

Stabilizing deep neural networks

Residual Connections

Also known as **skip connections**, these add the input of a layer to its output.

$$\text{Output} = \text{Layer}(\text{Input}) + \text{Input}$$

- **Knowledge Preservation:** Prevents information loss from earlier layers
- **Vanishing Gradient Solution:** Allows gradients to flow directly through the network

Why They Matter

In deep networks, input gets modified considerably by the time it reaches the last layer. Residual connections ensure **old information is not lost**.

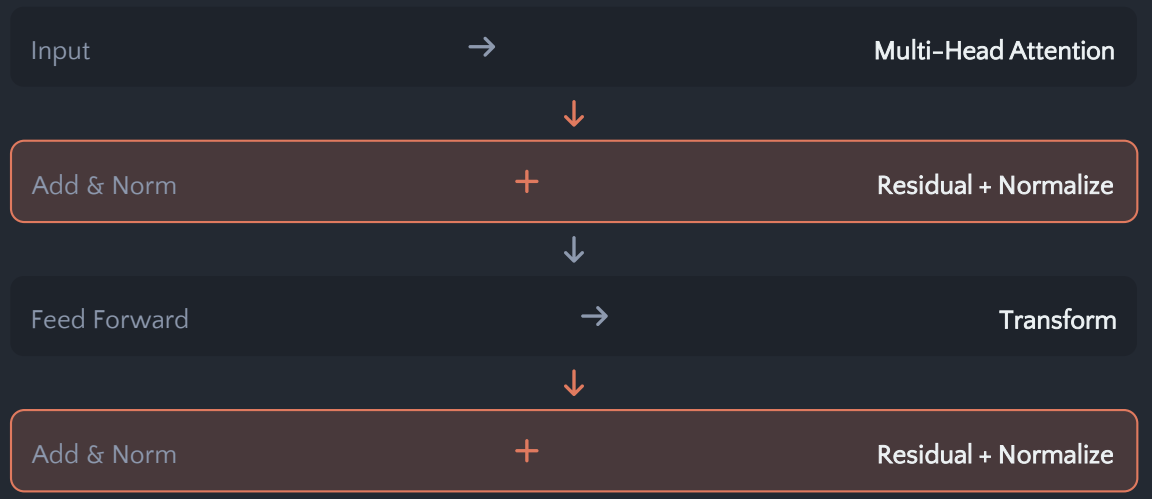
Layer Normalization

Applied **after every layer** to normalize the outputs.

Benefits:

- **Smoothens loss surface** → easier optimization
- **Stabilizes training** → faster convergence
- **Reduces internal covariate shift**

Architecture Integration



Together: Residual connections and layer normalization enable training of **very deep networks** (6+ layers) without degradation.

Types of Attention: A Comparison

Understanding different attention mechanisms

Simple Attention

(Encoder-Decoder)

Decoder attends to encoder outputs.

Example:

When generating "love" in translation, focus on "love" in input

Self-Attention

(Within Same Sequence)

Each word attends to all other words in the same sentence.

Example:

In "The animal didn't cross... because it was tired", "it" attends to "animal"

Multi-Head

(Parallel Attention)

Multiple self-attention mechanisms run in parallel.

Example:

One head links "it"→"animal", another captures "cross"→"street"

Detailed Comparison

Feature	Simple Attention	Self-Attention	Multi-Head
Definition	Focuses on relevant parts of encoder input while decoding	Sequence attends to itself	Multiple self-attention mechanisms in parallel
Used In	Seq2Seq models (Bahdanau, Luong)	Transformers (Encoder & Decoder)	Core part of Transformer layers
Input Scope	Decoder attends to encoder outputs	Each token attends to all tokens in same sequence	Each head attends to different subspaces
Purpose	Helps decoder align with encoder states	Builds context-aware representations	Increases model capacity for different patterns
Output	Weighted sum of encoder hidden states	Weighted sum of input embeddings	Concatenation of all heads + linear transform

KEY TAKEAWAYS

Transformers Revolutionized Sequence Processing

Parallel Processing

Replaced recurrence with attention, enabling all tokens to be processed simultaneously for dramatic speed improvements.

Attention Mechanism

The Query-Key-Value framework allows models to focus on relevant information, capturing complex relationships.

Multi-Head Power

Multiple attention heads run in parallel, each capturing different types of relationships for richer representations.

Long-Range Dependencies

No distance limitation - words at opposite ends of a sequence can directly attend to each other.

The Architecture That Changed Everything

From machine translation to GPT, BERT, and beyond

Thank you for learning!